

LLM Inference Engine Primer

Day 3 — domain background

Prep materials — Anthropic Fellows OA

August 2026

Contents

LLM Inference Engine Primer	1
1. What an inference engine does	1
2. Prefill vs. decode — the core asymmetry	1
3. KV cache	2
4. Batching	2
5. Scheduling policies	3
6. Load balancing across replicas	3
7. Metrics you might have to compute	3
8. Vocabulary cheat sheet	3
9. How this maps to the OA	3
10. Further reading (optional, ~1-2 hrs total)	4

LLM Inference Engine Primer

A from-scratch guide to the concepts behind the “inference engine” style OA problem: prefill/decode, KV cache, batching, and load balancing. Everything here is simulation-level — no GPUs or ML math needed, just the mechanics an OA would ask you to model.

1. What an inference engine does

When you send a prompt to an LLM server, the server:

1. **Tokenizes** the prompt into a sequence of tokens.
2. **Prefill**: runs the model over *all prompt tokens at once* to build internal state.
3. **Decode**: generates output tokens *one at a time*, each step feeding the previous token back in.
4. Streams tokens back until it hits a stop condition or `max_tokens`.

An “inference engine” (vLLM, TGI, TensorRT-LLM) is the scheduler + memory manager that runs many such requests concurrently on fixed hardware.

2. Prefill vs. decode — the core asymmetry

	Prefill	Decode
Input	Whole prompt (N tokens)	1 token per step
Cost	Proportional to prompt length; compute-bound	Cheap per step; memory-bandwidth-bound
Happens	Once per request	Once per generated token

	Prefill	Decode
Latency metric it drives	TTFT (time to first token)	ITL / TPOT (inter-token latency)

Key mental model: **prefill is a big one-shot batch job; decode is a long stream of tiny jobs.**
A request's total time = `prefill_time(prompt_len) + num_output_tokens x decode_step_time`.

In OA simulations this usually becomes something like:

```
prefill_time = ceil(prompt_tokens / prefill_rate) # e.g. tokens per tick
decode_time  = output_tokens * decode_cost      # e.g. 1 token per tick
```

Watch the units: per-tick rates vs. per-token costs, and whether time is discrete ticks or continuous.

3. KV cache

During prefill, the model computes a key/value tensor for every token; decode steps reuse them instead of recomputing. This is the **KV cache**.

What matters for a simulation:

- KV cache size grows linearly with sequence length (prompt + generated so far).
- It lives in GPU memory, which is finite → this is *the* scarce resource. A GPU can only host as many concurrent requests as its memory allows.
- When a request finishes, its cache is freed.

Typical OA modeling: each request consumes `prompt_len + tokens_generated_so_far` units of a per-GPU memory budget. Admission = only start a request if it fits (sometimes: if its *maximum possible* footprint fits, i.e. `prompt_len + max_new_tokens`).

vLLM's innovation, **PagedAttention**, allocates the cache in fixed-size blocks (like OS virtual memory pages) so memory isn't wasted on over-reservation. An OA might have you allocate/free blocks: `blocks_needed = ceil(seq_len / block_size)`.

4. Batching

GPUs are efficient when processing many requests per step.

- **Static batching**: collect B requests, run them together, wait until *all* finish. Simple, but a short request waits on the longest one.
- **Continuous (in-flight) batching**: at every decode step, the batch is recomposed — finished requests exit immediately, waiting requests join immediately. This is what modern engines do.
- **Chunked prefill**: long prefills are split into chunks and interleaved with decode steps of other requests, so one huge prompt doesn't stall everyone's token stream.

Simulation version of continuous batching, per tick:

```
for tick in count():
    finished = [r for r in running if r.done()]
    for r in finished: free_memory(r); running.remove(r)
    while queue and fits(queue[0]): # admission policy
        running.append(queue.popleft())
    for r in running: r.step()      # each generates 1 token
```

Edge cases OAs love: what happens on the exact tick a request finishes (does its memory free before or after admission?), tie-breaking among waiting requests (FIFO? shortest-first? priority?), and whether a request arriving at tick t can start at tick t.

5. Scheduling policies

You may be asked to implement one or more of:

- **FCFS / FIFO** — simplest; head-of-line blocking when a long request is first.
- **Shortest-Job-First** — minimizes average latency; needs a job-length estimate (prompt length or max tokens).
- **Priority scheduling** — requests carry a priority; higher preempts or jumps queue. Define tie-breaks (usually arrival order, then request id).
- **Preemption** — under memory pressure, evict a running request (drop its KV cache), re-queue it, and later *recompute* its prefill. Track wasted work.

6. Load balancing across replicas

With multiple GPU workers/replicas, a router assigns each request to a worker:

- **Round robin** — rotate through workers regardless of load.
- **Least connections / least outstanding requests** — send to the worker with fewest active requests.
- **Least load** — same idea but weighted by token counts or memory in use.
- **Session/prefix affinity** — route requests sharing a prompt prefix to the same worker so they can reuse cached KV (prefix caching).

OA gotchas: define “load” exactly as the README does (requests? tokens? memory?), tie-break deterministically (usually lowest worker index), and re-read whether assignment happens at arrival time or at dispatch time.

7. Metrics you might have to compute

- **TTFT** — arrival (or admission?) to first output token.
- **Latency / turnaround** — arrival to completion.
- **Waiting time** — arrival to start of service.
- **Throughput** — tokens (or requests) per unit time.
- **Utilization** — busy ticks / total ticks per worker.

Always check: inclusive or exclusive tick boundaries, and integer vs. float division.

8. Vocabulary cheat sheet

- **Token**: unit of text (~0.75 words).
- **TTFT / TPOT / ITL**: time to first token / time per output token / inter-token latency.
- **KV cache**: stored attention keys/values enabling cheap decode.
- **PagedAttention**: block-based KV cache allocation (vLLM).
- **Continuous batching**: per-step batch recomposition.
- **Chunked prefill**: interleaving prefill pieces with decode.
- **Prefix caching**: reusing KV cache across requests with a shared prompt prefix.
- **Speculative decoding**: small model drafts tokens, big model verifies (unlikely in an OA sim, but know the term).
- **Preemption / eviction**: dropping a running request’s cache to free memory.

9. How this maps to the OA

Reported OA shape: a pre-built codebase (classes for requests, workers, maybe a clock), a README describing levels, and functions for you to fill in — e.g. “implement the scheduler step” or “implement the load balancer’s pick_worker”. The algorithms are easy (queues, heaps, dicts, simulation loops). The difficulty is spec archaeology:

1. **Read the tests first** if visible — they are the real spec. README examples may not match the code; trust the tests, then the class signatures, then the README, in that order.
2. Nail down the **tick model**: what happens within one time step, and in what order (free memory → admit → step? or admit → step → free?). Get this from examples/tests.
3. Nail down **tie-breaking** everywhere a choice exists.
4. Get level 1 passing fast — levels build on each other, and partial credit clearly counts (people advanced with 120/600).

10. Further reading (optional, ~1-2 hrs total)

- vLLM paper/blog on PagedAttention and continuous batching.
- “LLM inference explained” style posts covering prefill/decode and KV cache.
- Orca paper (continuous batching origin) — skim the scheduling section only.